Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3605098.3636068

RESEARCH-ARTICLE

Elysia: Optimizing JavaScript Web Framework

CHAYAPATR ARCHIWARANGUPROK, University of the Thai Chamber of Commerce, Bangkok, Thailand

KONGKEIT KHUNPANITCHOT, University of the Thai Chamber of Commerce, Bangkok, Thailand

PHOOMPARIN MANO

MANACHAI TOAHCHOODEE, University of the Thai Chamber of Commerce, Bangkok, Thailand

Open Access Support provided by:

University of the Thai Chamber of Commerce

PDF Download
3605098.3636068.pdf
06 March 2026
Total Citations: 0
Total Downloads: 849

Published: 21 May 2024

Citation in BibTeX format

SAC '24: 39th ACM/SIGAPP Symposium on Applied Computing
April 8 - 12, 2024
Avila, Spain

Conference Sponsors:
SIGAPP

SAC '24: Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing (April 2024)
https://doi.org/10.1145/3605098.3636068
ISBN: 9798400702433

Elysia: Optimizing JavaScript Web Framework

Chayapatr Archiwaranguprok
The University of the Thai Chamber of Commerce
Bangkok, Thailand
pub@from.pub

Kongkeit Khunpanitchot
The University of the Thai Chamber of Commerce
Bangkok, Thailand
saltyaom@gmail.com

Phoomparin Mano
Creatorsgarten
Bangkok, Thailand
poom@poom.dev

Manachai Toahchoodee
The University of the Thai Chamber of Commerce
Bangkok, Thailand
manachai_toa@utcc.ac.th

ABSTRACT

As JavaScript and its backend web frameworks garnered mainstream adoption and played a pivotal role in today's industry, challenges of runtime performance, developer experience, and type safety are potential hindrances to scaling and maintainability.

This paper suggests strategies for web frameworks in JavaScript to provide a balanced focus between performance and convenience by optimizing the performance of potentially expensive ergonomic features by utilizing JavaScript's interpreted nature and Just-In-Time compilation through the use of static analysis and dynamic code generation techniques in conjunction with adopting TypeScript's type system to provide an improved developer experience.

The reference implementation of the optimization strategies and design for developer experience is illustrated through the implementations in the Elysia JavaScript web framework, in which the performance is evaluated against non-optimized code and benchmarked against other real-world backend web frameworks.

CCS CONCEPTS

• **Information systems → RESTful web services**; • **Software and its engineering → Software libraries and repositories**.

KEYWORDS

JavaScript, RESTful API, Static Analysis, Optimization, Type Safety

ACM Reference Format:
Chayapatr Archiwaranguprok, Kongkeit Khunpanitchot, Phoomparin Mano, and Manachai Toahchoodee. 2024. Elysia: Optimizing JavaScript Web Framework. In *Proceedings of ACM SAC Conference (SAC'24).* ACM, New York, NY, USA, Article 4, 8 pages. https://doi.org/10.1145/3605098.3636068

1 INTRODUCTION

In the last ten years, JavaScript has gained mainstream adoption as a programming language for backend web development. While JavaScript was designed for use in the browser, the emergence of JavaScript runtimes and frameworks such as Node.js and Express has enabled JavaScript to be widely used [3].

According to the Stack Overflow developer survey 2023 [24], JavaScript continues its eleventh-year streak as the most used programming language, used by 63.61% of 87,585 survey respondents. Backend frameworks developed in JavaScript are among the most popular in the "web technologies" category, with the Node.js runtime in the first place, used by 42.65% of 71,802 respondents, and the Express framework in fourth place by 19.28% of respondents, which is the most popular compared to frameworks in other languages, such as C#'s ASP.NET Core (16.57%), Python's Flask (12.16%), Java's Spring Boot (11.95%), and PHP's Laravel (7.58%). The statistics observed are similar to surveys in previous years, as shown in Figure 1.

However, when comparing the performance of web frameworks in JavaScript to other languages, the performance overhead of existing frameworks are significant. According to TechEmpower's performance benchmark on Round 21 [9], Express is ranked 113 out of 137 in composite test scores, while Nest is ranked 85 and Fastify at 76. This presents a challenge for optimizing the backend web framework in JavaScript.

Nevertheless, as Donald Knuth famously wrote, "Premature optimization is the root of all evil" [20], a gain in performance should not incur the cost of lower developer productivity and code maintainability in the long run. Given that, more aspects of framework design than optimization should be considered together. The aim should be to harmonize the core tenets of performance, developer experience, and type safety.

In search of achieving a balance, this paper introduces the Elysia framework, a TypeScript web framework serving as a practical reference implementation of optimization strategies and developer experience-centric design that addresses the challenges mentioned above. In addition, the paper assesses the optimization's performance compared to non-optimized code and compares it against other real-world backend web frameworks.

2 RELATED WORKS

2.1 TypeScript

TypeScript is an open-source programming language developed by Microsoft as a strict syntactical superset of JavaScript. TypeScript's type system enables the language compiler to detect many JavaScript bugs during compile time [13].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SAC '24, April 8–12, 2024, Avila, Spain
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0243-3/24/04…$15.00
https://doi.org/10.1145/3605098.3636068

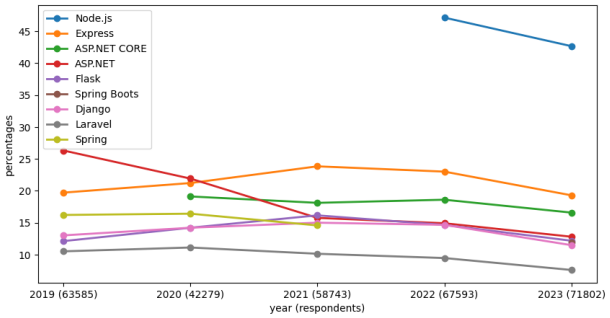

Figure 1: Backend Web Framework Usages Statistic from Stack Overflow Survey 2013–2019

TypeScript's type system improves developer experience and productivity by providing type-checking and autocompletion for developers. TypeScript also improves code maintainability by providing clear interfaces in type annotations, communicating expectations about data types to reduce misunderstandings between developers [5]. It makes refactoring or extending code easier without introducing subtle bugs, such as undefined values or type coercion [19]. As TypeScript is a superset of JavaScript, it can be compiled into JavaScript and utilized in frontend and backend applications.

2.2 JavaScript Runtimes

JavaScript runtimes integrate JavaScript engines with components such as the filesystem and HTTP server to create an environment for backend web development [39]. Node.js, a JavaScript runtime introduced in 2009, was pivotal in promoting the adoption of JavaScript on the server side. Node.js utilizes the V8 JavaScript engine [4], providing an event loop and APIs for non-blocking I/O [34].

Modern JavaScript runtimes such as Bun, Deno, Netlify Functions, and Vercel Functions are developed to improve runtime performance, developer experience, and ease of use [7, 25]. Elysia.js is primarily optimized for Bun, a runtime introduced in 2022 built with JavaScriptCore engine in the Zig language [31].

2.3 Stages of JavaScript Web Frameworks

JavaScript backend web frameworks provide a structured environment with pre-defined functions and utilities for building web applications, such as handling HTTP requests and managing databases [3]. They help developers follow best practices, save time, and maintain code consistency. A number of web frameworks are created for JavaScript, with varying designs and trade-offs.

Logically, backend web frameworks have a minimum of two stages: initialization and request handling. At initialization, the framework constructs the routing table by associating the HTTP method and path matchers with the route handler, registers the middleware functions [35] to process the request, and starts an HTTP server to accept requests. Upon each request, the framework would then parse the request into objects (e.g., header, body, cookie), resolve the matching route, forward the parsed data into each middleware layer, and then send the request back to the client.

2.4 End-to-end Type Safety

End-to-end type safety is a practice that aims to ensure type safety across the entire application stack, e.g., the frontend, the backend, and the database [1]. It aims to reduce runtime errors at every stage and improve developer productivity by providing consistent type inference and type checking across the application.

While RESTful APIs are often used to communicate between clients and servers [27], they have the drawback of not enforcing type safety by default. In addition, the types in client, server, and database schemas are often defined separately, which leads to inconsistencies and errors. For instance, when there are changes to the types in the backend, the client cannot statically detect the changes until runtime, leading to runtime errors.

Various technologies such as SOAP, Apache Thrift, JSON-RPC, gRPC, and GraphQL have been developed for achieving end-to-end type safety, where schemas are defined once to describe the interfaces of the API, then code generation tools generate typed client and server stubs [12, 17, 26]. However, the tooling and build steps required to configure code generation often add extra complexity.

2.5 JavaScript Optimization Tools

Several tools are being developed in the JavaScript ecosystem to enhance the performance of JavaScript code, including Google Closure Compiler[14] and Prepack[11]. These tools analyze JavaScript code and apply optimizations such as compile-time evaluation, dead code elimination, and function inlining.

However, this approach requires additional configuration in the application's build process, and application developers are tasked with configuring and testing the optimizations to ensure the tools and optimization flags function correctly with their source code.

In contrast, this paper presents a runtime-based method using Just-In-Time (JIT) compilation, eliminating the need for additional build steps. This approach offers a means to apply optimizations in a targeted manner without requiring extra configuration by end-users, as the web framework possesses the domain knowledge necessary to optimize functions safely.

3 DESIGN OF ELYSIA FRAMEWORK

Instead of making trade-offs between performance and ease of use, we can optimize the performance of expensive ergonomic features without sacrificing one another by leveraging the behavior of JIT compilation.

Elysia aims to provide a balanced focus on performance and convenience by building ergonomic features such as request parsing, documentation generation, end-to-end type safety, and strict validation via schema and applying optimization strategies such as static analysis and dynamic code generation via JIT compilation to eliminate their performance footprint.

To provide compatibility across runtimes, as there are multiple modern runtimes such as Bun and Deno being developed to improve performance, as well as to provide support for edge environments to decrease network latency. Elysia implements the web-interoperability community group (WinterCG [38]) specifications, which is a standard implemented by multiple modern web

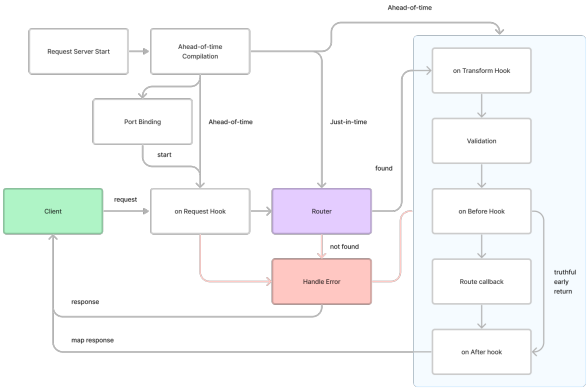

Figure 2: Elysia Lifecycle

runtimes, such as Bun, Deno, Netlify Edge, and Vercel Edge runtime, providing a single unified interface for HTTP request handlers across runtimes.

Elysia's contributions can be described as follows: dynamic code generation in JavaScript frameworks (3.1, 3.2, 3.5), conditional request parsing (3.3, 3.4), reducing the overhead of JavaScript (3.6 – 3.8), and applying typed schemas to achieve type safety and convenience (3.9 – 3.12).

3.1 Applying Ahead-of-Time Compilation

Elysia utilizes Ahead-of-Time (AOT) compilation to take advantage of the JavaScript engine's JIT compiler [40] in the framework initialization stage as described in section 2.3.

When an Elysia application initializes, the framework iterates over lifecycle hooks to dynamically construct the handler functions to optimize the request handling for individual routes, invoked when the framework receives an HTTP request. This is done by introspecting the source code of the lifecycle hook functions by using the `toString` method on route functions, which allows Elysia to determine which context argument is referenced. The rationale behind this process is explained in section 3.3 and 3.4.

Lifecycle hooks allow users to hook into events in the request pipeline to implement additional behavior, such as custom body parsers and authorization guards. They are invoked on each stage of the request pipeline, such as when the HTTP request is accepted, parsed, transformed, handled, errored, and sent to the client, and can be configured to either apply to specific endpoints or upon every request. They differ from middlewares in that they are invoked at specific stages of the request pipeline, while middlewares are typically only invoked before and after the request is handled.

Then, Elysia constructs a radix tree [23] for storing parametric routes and uses dynamic code generation to compile a function for matching against static routes using switch statements, which is covered in detail in section 3.7.

3.2 Introspection and Code Generation

Static analysis is a technique used to analyze source code without execution. It is often used to detect errors and security vulnerabilities [2] and identify code quality issues [30]. It can also identify performance bottlenecks to perform performance optimizations conditionally, such as dead code elimination [16] and code generation.

As JavaScript is an interpreted language, the function's source code can be statically analyzed at runtime through the usage of the `toString` method of `Function` instances, which returns a string representation of the code [15]. In addition, the `Function` constructor enables the JavaScript engine to perform a JIT compilation of the function at a runtime based on the string representation of the source code (Listing 1).

By leveraging the dynamic compilation capability of the `Function` constructor in conjunction with the introspection capability of `toString`, JavaScript frameworks can statically analyze the code for usage patterns, generate and compile an optimized support code based on the usage pattern, then evaluate the code at runtime. In the context of JavaScript frameworks, this can be used to understand how the user's functions use the features of the framework and alter the framework's behavior according to the user's code.

This technique enables users to dynamically generate supporting code, such as HTTP request parsers and lifecycle handlers, to reduce the amount of code required to be written by the framework's users to improve developer experience while tailoring it to the user's code to improve the performance at the same time. This technique will be explored in depth in further sections, where we apply the technique to HTTP request parsing in section 3.4 and to runtime optimizations in section 3.6 through 3.8.

```
function generate(method: Function) {
  const source = method.toString()

  if (doesFunctionReference(source, 'query'))
    return new Function(...)
}

```

Listing 1: Introspection and Dynamic Code Generation

3.3 Request Parsing Overhead

HTTP frameworks must parse the HTTP request for use in the route handler, such as extracting the HTTP headers, cookies, query strings, URL parameters, and various types of request bodies. For instance, Express and Fastify parse the headers and query strings and expose them to the route handlers by default, while parsing the body and cookie are handled by external packages such as multer and cookie-parser for Express and fastify-multipart and fastify-cookie for Fastify, as shown in Listing 2.

```
// Express
app.use(require('cookie-parser')())
app.post('/', (req, res) => res.send(req.cookies.session))

// Fastify
fastify.register(require('@fastify/cookie'))
fastify.post('/', (req, res) => res(req.cookies.session))
```

Listing 2: Registering and Using Middlewares

While request parsers improve the developer experience by reducing the amount of code required by the developer for parsing requests, they incur two significant performance penalties as demonstrated in Listing 2: eagerly parsing every portion of the request, and dynamically detecting the content type.

(1) The framework parses everything upfront as the framework does not know which will be referenced ahead of time, even though each API endpoint utilizes different portions of the HTTP request. For instance, an API endpoint for uploading a file may not reference the query parameters, but the framework will parse it nevertheless. By parsing the information that the route does not use, this incurs an overhead, as shown in the benchmarks in section 4.
(2) The request body is parsed for each request by getting the `Content-Type` header and applying the appropriate request body-parser, as the framework does not know about the expected content type ahead of time. This can incur a slight performance overhead. Alternatively, the developer can choose to specify the request parser for each endpoint, which can be inconvenient. Section 3.9 explores this issue further through the use of schemas for parsing and type inference.

3.4 Conditional Request Parsing

In order to eliminate the performance overhead of over-parsing requests, static analysis techniques are employed to identify which portion of the request is being used by the route handler, such as whether the route handler references the request body, query parameters, headers, or cookies. Elysia then conditionally constructs the handler function that only parses the portion of the referenced request based on its usage pattern.

In addition, Elysia allows developers to specify either the content type of the request or the strongly typed schema ahead of time to eliminate the overhead of content type detection, which has the added benefit of providing an improved developer experience by providing a strong type system [28], i.e., automatically parsing and validating content type and providing end-to-end type safety and type autocompletion. The use of strongly-typed schemas for parsing will be explored further in section 3.9.

For each introspection, the framework incrementally constructs the string literal of the resulting handler function, which includes the support code necessary for parsing the request and invoking the method provided by the user (Listing 3), which ultimately gets JIT-compiled into a function using the `Function` constructor. This introspection also allows Elysia to perform additional optimizations to reduce the overhead of the JavaScript engine, which will be explored further in 2.2, runtime-based optimizations.

```
if (doesFunctionReference(source, 'headers'))
  wrapper += `c.headers = c.request.headers.toJSON()`

if (doesFunctionReference(source, 'query') )
  wrapper += `c.query = parseQuery(c.request.url)`
```

Listing 3: Conditionally Applying Parsers

3.5 Detecting Arguments Reference

In order to determine whether the route function references the specific arguments of the HTTP request, such as `ctx.cookie`, `ctx.query` or `ctx.headers`, we can pattern-match the function's arguments.

The logic for detecting whether a JavaScript function references an argument must consider access by dot notation, bracket notation, object destructuring, or reference to an aliased variable by matching against the following scenarios.

(1) **The function does not accept any arguments**: The function's arguments between the first pairs of parentheses of a function literal must be empty.
(2) **The function uses object destructuring to extract the arguments**: The list of arguments must begin with an opening brace, and the target argument must begin with either an opening brace or a comma.
(3) **The function references the arguments via dot or bracket notion**: The function's string literal matches against the `<ARGUMENTS>(.<KEYWORD>|\["<KEYWORD>"\])` regular expression.
(4) **The function references the arguments by an alias to another variable**: Identify potential aliases for the function argument using a regular expression. Then, use the `{.*?} = (<ALIASES>)` regular expression to extract the variables destructured from aliases.

After identifying whether the route handler's string literal references each request argument, as exemplified by the function in Listing 4, we utilize dynamic code generation to construct the lifecycle hook functions and the corresponding switch-case mapping outlined in 3.7.

```
function doesFunctionReference(fn, key) {
  let args = fn.slice(fn.indexOf('(') + 1, fn.indexOf(')'))
  if (!args) return false

  if (args.charCodeAt(0) === 123)
    return args.includes('{ ' + key) ||
           args.includes(', ' + key)

  let ref = new RegExp(`${args}(.${key}|\\["${key}"\\])`)
  if (fn.match(ref)) return true

  let alias = new RegExp(`(\\w+) = context`, 'g')
  return isAliasReferenced(fn.matchAll(alias))
}
```

Listing 4: Detecting Argument References

3.6 Inlining Property Assignments

Elysia's decorators' functionality allows users to add custom properties to the framework's context object via the `decorate` method, which is then merged with the framework's context objects and exposed to each lifecycle hook. This is useful for storing object instances, such as instances of database connection objects, or for storing values that are expensive to compute. This requires merging two JavaScript objects, one from the framework and another defined by the framework's user.

However, an application could contain many lifecycle hooks created by plugins and application code, and each lifecycle hook must construct its context object containing the information from the previous stage of the request pipeline. When benchmarking the context creation process, we found that using the object spread operator or the `Object.assign` method has a performance overhead in contrast with inlining the property assignment.

By using dynamic code generation, the framework can introspect the shape of the context object specified by the user upon application initialization, then generate an inline key-value mapping within the object to eliminate this overhead as per Listing 5.

```
for (let key of Object.keys(app.decorators))
  literal += `,${key}: app.decorators.${key}`
```

Listing 5: Generating Property Assignment

3.7 Inlining Static Route Matching

JavaScript frameworks use various data structures to store route mappings. For instance, Fastify stores route mappings as radix trees [36], while Express, Hapi, and Koa store the routes as JavaScript arrays [33, 37]. A route in web frameworks can usually be either static strings (`/users`), parametric strings containing URL parameters (e.g., `/users/:id/profile` containing the `:id` parameter), or regular expressions (e.g., `/\^\/users\/(\d+)\/profile$/`).

As Elysia's router supports both static routes and parametric routes, Elysia stores the static routes in a hash map for constant-time lookup, while, parametric routes are stored in Radix Trees.

However, as the performance of hash map lookup varies from one JavaScript engine to another, the JavaScript framework must conditionally apply optimizations using dynamic code generation, either by measuring the execution profile and then applying profile-guided optimizations (PGO) or dynamically generating the optimized code based on the runtime environment.

In particular, as switch map lookups can be faster than hash map lookups in some JavaScript engines, we can construct a string literal representation of a `switch` statement that maps the static routes to the corresponding handler functions at the AOT compilation stage. This is done by iterating over the hash map and then constructing a `case` statement for each route containing the call to route handler functions (Listing 6). The `switch` statement is then JIT-compiled into a function using the `Function` constructor.

```
for (let [path, {code, all}] of entries(map))
  switchMap += `case '${path}':\nswitch(request.method) {\n${code}\n${all ?? 'default: break map'}}`
```

Listing 6: Generating Switch Statements

3.8 Conditional Await Expressions

Lifecycle hooks such as `onRequest` and `onParse` are used by plugins and route handlers to add and modify behaviors of HTTP routes, such as adding session validation, performance tracing, and error handling. The framework is responsible for executing these lifecycle hook functions at each stage of the request processing. However, lifecycle hooks can either be synchronous or asynchronous functions returning a `Promise`. JavaScript's `await` expression suspends the execution of asynchronous functions until the promise settles. Specifically, the runtime defines a continuation function to be called once the promise settles, which gets added to the microtask queue for execution before the next tick of the event loop [21].

However, this execution process introduces an overhead when adding the continuation function to the microtask queue. A promise that takes a long time to settle will block other code queued on the current event loop. In addition, asynchronous functions which contain multiple await expressions require additional stack spaces to store the continuation callbacks [8].

The use of multiple `await` statements can reduce the performance of the code even if the awaited functions are synchronous. This results in performance overhead, especially when hundreds of synchronous lifecycle hooks could be inserted by plugins for a given route. We can eliminate this by detecting whether a lifecycle hook function is asynchronous by checking whether the function constructor is an `AsyncFunction`, then conditionally append the `await` keyword to the function string literal at the initialization stage, as demonstrated below.

```
let isAsync = (fn) => fn.constructor === 'AsyncFunction'

wrapper += isAsync(hooks.beforeHandle[i])
  ? `let ${name} = await beforeHandle[${i}](c);\n`
  : `let ${name} = beforeHandle[${i}](c);\n`
```

Listing 7: Conditional Await Expressions

3.9 Request Parsing via Schemas

Elysia provides the option for users to either specify the content type expected by a route, such as `arrayBuffer`, `json`, `urlencoded`, and `formdata`, or to specify a strongly-typed schema. For instance, if a route is specified to respond with a `formdata` type or the response schema references the `Files` type, the framework will parse the files from the request into a `Files` object for the developer to use.

At the AOT compilation stage, Elysia appends and compiles the request parser code for each lifecycle hook based on the provided response or schema type. For instance, the framework appends the code for extracting the URL-encoded query into a plain object when the `urlencoded` response type is set and appends the code to transform a multipart form data into a plain object when the `formdata` response type is set.

Similarly, when a strongly-typed schema is present, the framework introspects the schema to parse the request body, such as applying the form data parser when the schema references the `Files` type or applying the query parser when the schema references the `URLEncoded` type. The framework provides these custom types, which are explained in detail in the next section.

This approach, as illustrated in Listing 8, provides the convenience of parsing the request according to the specified schema or response type and eliminates the performance overhead of detecting the content type and determining the request parser at runtime.

```
if (type === 'urlencoded' || schema.meta === 'URLEncoded')
  return `c.body = parseQuery(await req.text())`
```

Listing 8: Conditional Parsing with Schemas

3.10 Custom Types for Schemas

Elysia uses the schema's type information both for runtime usages, such as parsing the request's query parameters and body, generating the Swagger documentation, and for providing compile-time type inference and type checking for improved developer experience.

The framework implements this using the `@sinclair/typebox` library [29] for schema definition and inference. Elysia exposes a set of custom types, which provides automatic request parsing at runtime and automatic type inference at compile time.

For instance, the `Numeric` type parses integers and floats, the `URLEncoded` type parses the URL-encoded query string from `application/x-www-form-urlencoded` requests into the `Record`, the `File` and `Files` type parses the multipart form data from `multipart/form-data` requests into the `File` and `File[]` as shown in listing 9, and the `Cookie` type parses the cookie header from requests into the `Record` type.

```
const $files = TS<File[], Opts.Files>('Files', validate)
const files: (options: Opts.Files) =>
  Type.Transform(Type.Union([$files(options)]))
    .Decode((v) => Array.isArray(v) ? v : [v])
type Files = typeof files
```

Listing 9: Type Validations and Transformations

At runtime, the schema definition is traversed with a recursive function to extract the metadata at the AOT compilation stage, which then returns a list of properties with attached metadata, such as the `Numeric` type defined in the above example.

Notably, the strongly-typed schema is used to generate OpenAPI schemas and Swagger documentation. This is done by traversing the route mapping object and transforming the schema definitions into OpenAPI schemas [22], which is then exposed to Swagger to provide documentation for the API without user intervention.

3.11 Designing APIs for Type Inference

Elysia's methods, such as `state`, `decorate`, `derive`, `hook`, `model`, and `error`, are designed to provide type inference for the developer by dynamically constructing types based on the arguments given by the user, which are then used for inference in the rest of the application instance. For instance, the `derive` is used to derive new context values based on existing context values.

When the state, decorate and derive methods are used, the type of these values can be referenced for type inference in hooks at compile-time. In Listing 10, `store.version`, `getDate`, and `authorization` arguments in the `get` handler are inferred from the earlier `state`, `decorate`, and `derive` calls, respectively. This is possible through the use of generic types and generic type parameters in TypeScript. The methods inject the type information into the `ElysiaInstance<Instance>` generic type, namely the `Instance.store` and `Instance.request` type parameters.

This technique of using generic types and type parameters for type inference is used extensively in Elysia to provide complete type inference for complex use cases, such as deeply nested routes, route guard handlers, schema and model definitions, extensible plugin systems, and typed error states.

```
app.state('foo', 1)
   .decorate('bar', Date.now)
   .derive(c => ({baz: c.headers.get('baz')}))
   .get( '/', (c) => c.foo + c.bar() + c.baz)
```

Listing 10: Type Inference In Elysia APIs

3.12 End-to-End Type Safety

Elysia achieves end-to-end type safety without the complexity of code generation by importing TypeScript type definitions from the server to the client via local or URL imports and then transforming the server's type interfaces into the desired client interface. We provide Eden Treaty, a fetch client for Elysia applications using JavaScript's proxy objects in conjunction with TypeScript's type transformations in place of code generation.

```
import type {App} from './server'

let app = edenTreaty<App>(endpoint)
await app.users[12].post()
```

Listing 11: Importing Server Types into Client

TypeScript's type-level constructs, such as generics, conditional types, recursive types, and mapped types, can transform one data type into a different type by recursively enumerating the properties of a type and performing type-level transformations.

Eden Treaty utilizes this functionality by creating a generic type to recursively enumerate over the server's internal schema type, transforming the route schema on the type level into an object-like representation of the Elysia application. This benefits strong type safety as the type definitions are always up-to-date and in sync with the server, as the type is inferred from the server's type.

```
Paths extends [infer P, ...infer Rest]
  ? {[Key in Prefix as Prefix extends `:${string}`
      ? (string & {}) | number | Prefix
      : Prefix]: Parse<Schema, Rest, `${Carry}/${Key}`>}
  : Schema[Carry extends '' ? '/' : Carry] extends ...
```

Listing 12: Type-Level Transformations with Generics

Eden Treaty's client implementation constructs a JavaScript proxy object for trapping function calls within the proxy object. When the client invokes a method defined in Eden Treaty's fetch client, such as `app.users[24].get()`, the call signature (e.g., `users[24].get`) is intercepted by the proxy object's `[[Call]]` handler. Eden Treaty registers a function call handler on the proxy object, as demonstrated in Listing 13, to transform the call signature into RESTful routes via regular expressions (e.g., GET `/users/24`) or WebSocket subscriptions matching the derived type signature, then makes an HTTP request to the server based on the derived endpoint and specified parameters.

```
let createProxy = (domain, path) =>
  new Proxy(() => {}, {
    get: (_, key) => createProxy(domain, path + '/' + key),
    apply(target, _, [params]) {
      let i = path.lastIndexOf('/')
      let method = path.slice(i + 1).toUpperCase()
      let url = composePath(domain, path.slice(0, i))
      return createFetcher(url, method, params)
    },
```

```
})
```

Listing 13: Using Proxies for Method Interception

4 IMPACT ON STATIC ANALYSIS AND DYNAMIC CODE GENERATION

We evaluate the effectiveness of performance optimization techniques based on static analysis and dynamic code generation by benchmarking the optimized version of the code with the control. The benchmarks are run on three JavaScript runtimes to demonstrate that the performance increases are not specific to any JavaScript runtime or engine but are generally applicable as these optimizations target the overhead present in the ECMAScript specifications. With that said, benchmarking across runtimes is vital as different JavaScript engines may still have varying underlying implementations or optimizations, resulting in different performance characteristics.

This benchmark is tested under the following conditions:

(1) Apple M1 Max, 32 Core GPU, 64GB RAM, 1TB SSD
(2) MacOS 14.0 (Sonoma)
(3) Runtimes: bun v1.0.3 / deno v1.37.1 / node v18.16.0

Per the benchmark, the optimization strategies employing dynamic code generation such as conditional request parsing (3.4), inlining property assignments (3.6), inlining static route matching (3.7), conditional await expressions (3.8), and request parsing via schemas (3.9), indicates a significant performance improvement.

Notably, the optimization techniques for inlining static route matching depend on the JavaScript runtime, as using switch statements has a significant performance improvement on Bun. At the same time, direct hashmap access is more performance on Deno. Therefore, the framework must use dynamic code generation to conditionally apply the optimizations from the execution profile.

In addition, the performance of request parsing using Elysia's request parser on Node.js is noticeably slower. This is due to the Node.js runtime not yet supporting the WinterCG fetch specifications; therefore, the benchmark is relying on a third-party request conversion library to transform Node's HTTP request into WinterCG's web standard request, and the framework is unable to apply the ahead-of-time request parsing optimizations in Node.js, as the request conversion process parses the entire request.

5 DISCUSSION AND FUTURE WORK

5.1 Elysia in practice

5.1.1 Benchmark. In addition to the evaluation presented in Section 4, we have included a benchmark of Elysia against other frameworks in JavaScript and various programming languages. This benchmark is intended to offer a real-world context where framework performance is assessed based on actual usage scenarios, irrespective of control variables such as language and environment.

To ensure impartiality in our results, we have abstained from conducting our own benchmarks, and presented data derived from a third-party test with control variables and methods determined by the tester. The results we present have been sourced from TechEmpower's Round 22 benchmark from 17 October 2023. The hardware setup includes three homogeneous Dell R440 servers, each equipped with an Intel Xeon Gold 5120 CPU, 32GB of memory, and an enterprise SSD. The servers are connected through a dedicated Cisco 10-gigabit Ethernet switch [10].

Table 1: Evaluation (operations/second)

Benchmark	Bun	Deno	Node.js
Conditional Request Parsing			
Ahead-of-Time	6,002,130	254,453	1,295
Dynamic	3,316,434	146,199	4,503
Inlining Property			
Inline	192,981,693	209,205,021	1,017,716,241
Spread	18,144,290	22,045,855	23,005,245
Inlining Static Route Matching			
Switch Statement	32,026,708	160,513,643	1,014,650,611
Nested HashMap	25,728,365	68,119,891	218,845,428
HashMap	23,894,456	241,545,894	1,007,574,601
Overhead of Await in Sync Functions			
Without Await	256,120,282	162,337,662	162,501,497
With Await	27,061,980	8,871,540	4,624,318
Request Parsing via Schemas			
Elysia Typed	2,365,646	136,799	5,050
Elysia Non Typed	1,553,367	130,890	5,296

Each framework underwent testing within its own Docker image, covering a range of test types such as JSON serialization, single query execution, and data updates. The benchmark's source code is publicly accessible on TechEmpower's GitHub repository [32].

Due to the extensive scope of frameworks assessed in the original result, the data presented in Table 3 is primarily derived from the Stack Overflow survey, as discussed in Section 1, and the three most widely used JavaScript backend web frameworks according to the State of the JavaScript 2020-2022 survey [6], which serves as a point of reference.

Table 2: Top JavaScript Backend Web Framework by Usages from the State of JavaScript Survey from 2020 to 2022

Year	Respondents	Express	Nest	Fastify	Koa	Hapi
2022	39,471	87.3%	30.2%	15.4%	8.3%	3.7%
2021	16,085	81%	17%	11%	-	-
2020	23,765	80%	13%	6%	16%	9%

5.1.2 Adoption. Regarding developer experience, it is objectively hard to perform a proper evaluation. However, we provide data on actual world usage for Elysia. As of September 2023, the framework has around 5,000 stars on its GitHub repository and around 10,000 downloads per week on npm, indicating a significant interest in the framework.

5.1.3 Using Elysia. The code for the framework is open-source on GitHub [18], and the package can be installed through the node package manager's (NPM) repository.

Table 3: Benchmark Results (requests/second)

Framework (Version)	Language	JSON	1-query	20-query	Fortunes	Updates	Plaintext	Weighted score	Rank
ASP.NET Core (8.0)	C#	1,042,029	392,709	25,329	363,344	18,197	7,014,298	6,143	15
Elysia (0.7)	TypeScript	667,582	267,566	24,538	174,261	19,263	2,454,631	4,093	32
ASP.NET Core (8.0) [MVC]	C#	546,836	249,450	21,735	158,187	16,144	2,406,637	3,593	39
Spring (3.0.0)	Java	236,259	147,907	15,932	24,082	7,131	506,087	1,507	88
Fastify (4.13.0)	JavaScript	275,038	94,664	7,986	71,782	3,432	415,600	1,208	102
NodeJS (18.12.1)	JavaScript	320,168	95,776	6,860	70,833	3,048	454,082	1,208	104
NestJS (6.11.11)	TypeScript	270,076	76,938	5,975	61,081	3,641	419,035	1,099	106
Express (4.17.1)	JavaScript	92,604	37,488	4,806	33,868	2,005	113,117	555	130
Laravel (10.39.0)	PHP	77,648	37,275	4,656	22,501	1,666	81,052	462	138
Django (3.2.23)	Python	177,099	19,032	1,623	14,707	871	300,170	413	143

5.2 Future Works

A potential possibility is to explore applying static analysis and dynamic code generation techniques to other languages, where the language allows introspection and transformation of the user code. For instance, the Rust programming language allows building procedural macros to transform a function's token stream to enable the creation of DSLs and custom syntaxes and generate code at compile time. This is achieved by parsing the token stream into an abstract syntax tree (AST), introspecting the usage of the argument from the AST, and then applying the techniques in section 3 such as conditional parsing, to improve the performance.

REFERENCES

[1] Sabin Adams. 2022. End-To-End Type Safety: What, Why and How. https://sabinadams.hashnode.dev/end-to-end-type-safety-what-why-and-how

[2] Tiago Brito, Mafalda Ferreira, Miguel Monteiro, Pedro Lopes, Miguel Barros, José Fragoso Santos, and Nuno Santos. 2023. Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages. http://arxiv.org/abs/2301.05097 arXiv:2301.05097 [cs].

[3] Ethan Brown. 2019. *Web development with node and express* (2 ed.). O'Reilly Media, Sebastopol, CA.

[4] Mathias Bynens. 2018. Celebrating 10 years of V8. https://v8.dev/blog/10-years

[5] Boris Cherny. 2019. *Programming TypeScript.* O'Reilly Media, Sebastopol, CA.

[6] Devographics. 2023. Stack Overflow Developer Survey 2023. https://survey.stackoverflow.co/2023

[7] Fernando Doglio. 2020. *Introducing Deno* (1 ed.). APress, Berlin, Germany.

[8] Dan Elphick. 2019. v8/src/execution/microtask-queue.h (@ec06bb6). https://source.chromium.org/chromium/chromium/src/+/main:v8/src/execution/microtask-queue.h;drc=ec06bb6ce5641cf65e400ec55b7421f87d04b999

[9] Tech Empower. 2022. Web Framework Benchmarks Round 21. https://www.techempower.com/benchmarks/#section=data-r21

[10] Tech Empower. 2023. Web Framework Benchmarks Round 22. https://www.techempower.com/benchmarks/#section=data-r22

[11] Facebook. 2017. Prepack. https://www.netlify.com/blog/intro-to-serverless-functions/

[12] Apache Software Foundation. 2023. Apache Thrift. https://thrift.apache.org

[13] Zheng Gao, Christian Bird, and Earl T. Barr. 2017. To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. In *Proceedings of the 39th International Conference on Software Engineering* (Buenos Aires, Argentina) *(ICSE '17)*. IEEE Press, 758–769. https://doi.org/10.1109/ICSE.2017.75

[14] Google. 2023. Google Closure Compiler. https://github.com/google/closure-compiler?tab=readme-ov-file

[15] Shu-yu Guo, Michael Ficarra, and Kevin Gibbons. 2023. *ECMA-262, ECMAScript®2023 Language Specification* (14 ed.). ECMA International. https://262.ecma-international.org/14.0/.

[16] Roman Haas, Rainer Niedermayr, Tobias Roehm, and Sven Apel. 2020. Is Static Analysis Able to Identify Unnecessary Source Code? *ACM Trans. Softw. Eng. Methodol.* 29, 1, Article 6 (jan 2020), 23 pages. https://doi.org/10.1145/3368267

[17] Kasun Indrasiri and Danesh Kuruppu. 2020. *GRPC: Up and running.* O'Reilly Media, Sebastopol, CA.

[18] Kongkeit Khunpanitchot. 2022. Elysia.js: Ergonomic Framework for Humans. https://github.com/elysiajs/elysia

[19] Sebastian Kleinschmager, Romain Robbes, Andreas Stefik, Stefan Hanenberg, and Eric Tanter. 2012. Do static type systems improve the maintainability of software systems? An empirical study. In *2012 20th IEEE International Conference on Program Comprehension (ICPC)*. 153–162. https://doi.org/10.1109/ICPC.2012.6240483

[20] Donald E. Knuth. 1974. Computer Programming as an Art. *Commun. ACM* 17, 12 (dec 1974), 667–673. https://doi.org/10.1145/361604.361612

[21] Matthew C. Loring, Mark Marron, and Daan Leijen. 2017. Semantics of Asynchronous JavaScript. *SIGPLAN Not.* 52, 11 (oct 2017), 51–62. https://doi.org/10.1145/3170472.3133846

[22] Darrel Miller, Jeremy Whitlock, Marsh Gardiner, Mike Ralphson, Ron Ratovsky, and Uri Sarid. 2021. *OpenAPI Specification v3.1.0.* the Linux foundation. https://spec.openapis.org/oas/v3.1.0.

[23] Donald R. Morrison. 1968. PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric. *J. ACM* 15, 4 (oct 1968), 514–534. https://doi.org/10.1145/321479.321481

[24] Stack Overflow. 2023. State of JavaScript. https://stateofjs.com/en-US

[25] Brittney Postma. 2022. Intro to Serverless Functions | Netlify. https://www.netlify.com/blog/intro-to-serverless-functions/

[26] Antonio Quiña Mera, Pablo Fernandez, José María García, and Antonio Ruiz-Cortés. 2023. GraphQL: A Systematic Mapping Study. *ACM Comput. Surv.* 55, 10, Article 202 (feb 2023), 35 pages. https://doi.org/10.1145/3561818

[27] Leonard Richardson, Mike Amundsen, and Sam Ruby. 2013. *Restful web APIs.* O'Reilly Media.

[28] Vlad Riscutia. 2019. *Programming with types.* Manning Publications, New York, NY.

[29] sinclairzx81. 2023. TypeBox: Json Schema Type Builder with Static Type Resolution for TypeScript. https://github.com/sinclairzx81/typebox

[30] Darko Stefanović, Danilo Nikolić, Dusanka Dakic, Ivana Spasojević, and Sonja Ristic. 2020. *Static Code Analysis Tools: A Systematic Literature Review.* 0565–0573. https://doi.org/10.2507/31st.daaam.proceedings.078

[31] Jarred Sumner. 2022. Bun. https://github.com/oven-sh/bun

[32] TechEmpower. 2023. TechEmpower Framework Benchmarks. https://github.com/TechEmpower/FrameworkBenchmarks

[33] Franky Tse. 2020. [NodeJS] Express source code — Router, Route, Layer. [NodeJS] ExpresssourcecodeâĂŤRouter,Route,Layer

[34] Malte Ubl. 2009. Ryan Dahl: Node.js, Evented I/O for V8 Javascript. https://www.jsconf.eu/2009/speaker/speakers_selected.html

[35] Shiju Varghese. 2015. HTTP Middleware. In *Web Development with Go.* Apress, Berkeley, CA, 99–120. https://doi.org/10.1007/978-1-4842-1052-9_6

[36] Tomas Della Vedova. 2023. find-my-way: A crazy fast HTTP router. https://github.com/delvedor/find-my-way

[37] Tomas Della Vedova. 2023. @hapi/call: Simple HTTP Router. https://github.com/hapijs/call

[38] Web-interoperable Runtimes Community Group 2023. *WinterCG Fetch Draft Community Group Report.* Web-interoperable Runtimes Community Group. https://fetch.spec.wintercg.org/.

[39] Allen Wirfs-Brock and Brendan Eich. 2020. JavaScript: The First 20 Years. *Proc. ACM Program. Lang.* 4, HOPL, Article 77 (jun 2020), 189 pages. https://doi.org/10.1145/3386327

[40] Nicholas C Zakas. 2010. *High Performance JavaScript.* O'Reilly Media, Sebastopol, CA.